Algorithmique

Notes de cours

Université Paris Diderot

Copyright © 2015 Sophie Laplante (Professeur), notes de Maxime Gourgoulhon (étudiant)

PUBLISHED BY CC

INFORMATIQUE.UNIV-PARIS-DIDEROT.FR

Licensed under the Creative Commons Attribution-NonCommercial 3.0 Unported License (the "License"). You may not use this file except in compliance with the License. You may obtain a copy of the License at `http://creativecommons.org/licenses/by-nc/3.0`. Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

First printing, November 2015

Table des matières

I Union-Find

1 Graphes 9
1.1 Définition 9
1.2 Représentations 9
1.2.1 Matrice d'adjacence 10
1.2.2 Liste d'adjacence 10
1.3 Chemins 10
1.4 Cycle 10
1.5 Connexe 10
1.6 Arbres 10
1.7 Arbre couvrants 10
1.8 Graphes valués 11
1.9 Algorithme de Kruskal (1956) 11

2 Cycles et composantes connexes 13
2.1 Problèmes 13
2.2 Application : Kruskal 13

3 L'algorithme : Union-Find 15
3.1 Définition 15
3.2 Exemple 15
3.3 Applications 15

3.4 Idée de l'algorithme 16
3.5 Retour sur Kruskal 16
3.6 Implémentation en Python 16
3.7 Complexité 17
3.8 $UNION$: Maîtriser la hauteur des arbres 17
3.9 $FIND$: Compresser les chemins 17

II Parcours d'arbres

4 Arbres binaires de recherche ABR 21
4.1 Complexité 21
4.2 Arbres binaires de recherche 21
4.3 Recherche dans un ABR 22
4.3.1 Preuve par récurrence sur $|A|$ 22
4.4 Insertion dans un ABR 23
4.4.1 Algorithme 23
4.4.2 Coût moyen d'insertion de n éléments dans un ABR vide 23

III Parcours de graphes

5 Parcours d'arbres k-aire 27
5.1 Structure de donnée utilisée 27
5.2 Parcours en largeur des arbres 28
5.3 Propriétés du parcours en largeur 28
5.4 Parcours en profondeur des arbres 29
5.5 Propriétés du parcours en profondeur des arbres 29

6 Parcours de graphes 31
6.1 Représentation des graphes (non orientés) 31
6.2 Parcours en largeur des graphes 31
6.3 Parcours en profondeur des graphes 33
6.3.1 Version récursive 33
6.3.2 Version itérative 33

IV Algorithmes de graphes

7 Algorithme de Kruskal 37
7.1 Implémentation de l'algorithme de Krukal 37
7.2 Correction de l'algorithme de Kruskal 37
7.2.1 Lemme d'optimalité 38

8 Algorithme de Prim 39
8.1 Implémentation de l'algorithme de Prim (version optimisée) 39
8.2 Complexité de l'algorithme de Prim 40

9 Les plus cours chemins : Dijkstra 41
9.1 Algorithme de Dijkstra 41
9.2 Preuve de l'algorithme de Dijkstra 42

10 Algorithme de Bellman-Ford 45
10.1 Idée 45
10.2 Implémentation en Python 46
10.3 Preuve 46

11 All pairs shortest paths 47
11.1 Dijkstra, plusieurs fois 47
11.2 Modification de Bellman-Ford 47
11.3 En doublant la longueur des chemins à chaque itération 48
11.4 Algorithme de Floyd-Warshall 49

12 Cycles négatifs 51
12.1 Algorithme pour détecter les cycles négatifs 51

Index 53

Union-Find

1 Graphes 9
1.1 Définition
1.2 Représentations
1.3 Chemins
1.4 Cycle
1.5 Connexe
1.6 Arbres
1.7 Arbre couvrants
1.8 Graphes valués
1.9 Algorithme de Kruskal (1956)

2 Cycles et composantes connexes 13
2.1 Problèmes
2.2 Application : Kruskal

3 L'algorithme : Union-Find 15
3.1 Définition
3.2 Exemple
3.3 Applications
3.4 Idée de l'algorithme
3.5 Retour sur Kruskal
3.6 Implémentation en Python
3.7 Complexité
3.8 $UNION$: Maîtriser la hauteur des arbres
3.9 $FIND$: Compresser les chemins

1. Graphes

1.1 Définition

Définition 1.1.1 — Graphe. Un graphe est constitué d'un ensemble de sommets S d'un ensemble d'arêtes $A \subseteq S \times S$ reliant les sommets. Les arêtes sont orientées (flèche) ou non orientées (segment).

Notation 1.1. *On note un graphe $G = (S, A)$ en français, $G = (V, E)$ en anglais.*

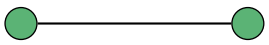


FIGURE 1.1 – Graphe non orienté

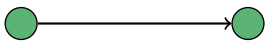


FIGURE 1.2 – Graphe orienté

Les sommets peuvent représenter des villes, des stations de métro, des sites sur des cartes géographiques, etc. Les arêtes peuvent symboliser des routes, des câbles, des liens, etc. Un graphe peut ainsi représenter des cartes géographiques, le web, les réseaux sociaux, des modèles 3D (animations), modèles physiques d'interactions...

1.2 Représentations

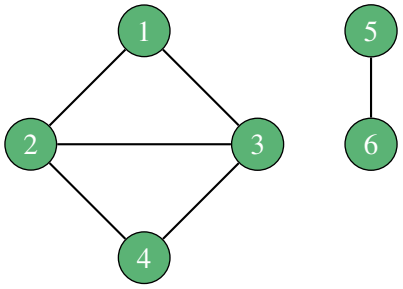


FIGURE 1.3 – Graphe non orienté

	1	2	3	4	5	6
1	0	1	1	0	0	0
2	1	0	1	1	0	0
3	1	1	0	1	0	0
4	0	1	1	0	0	0
5	0	0	0	0	0	1
6	0	0	0	0	1	0

FIGURE 1.4 – Matrice d'adjacence du graphe 1.3

1	2	3	
2	1	3	4
3	1	2	4
4	2	3	
5	6		
6	5		

FIGURE 1.5 – Liste d'adjacence du graphe 1.3

1.2.1 Matrice d'adjacence

Une matrice d'adjacence pour un graphe G à n sommets est une matrice de dimension $n \times n$ dont où l'élément à la position i, j vaut 1 s'il existe une arête reliant le sommet i au sommet j, 0 sinon.

1.2.2 Liste d'adjacence

Une liste d'adjacence est la liste des voisins de chaque sommets. C'est une représentation relativement compacte lorsqu'il y a peu d'arêtes.

1.3 Chemins

Définition 1.3.1 — Chemin. Un chemin de longueur k de u à v dans un graphe $G = (S, A)$ est une suite de sommets $u_0, u_1, \ldots u_k$ telle que $u_0 = u$, $u_k = v$ et on ne passe que par des arêtes de G. $\forall i \in [\![0, k]\!] \in A$

Définition 1.3.2 — Chemin simple. Un chemin simple est un chemin qui ne passe pas 2 fois par le même sommet.

Exemples sur la figure 1.3 :
— 1, 3, 2, 4 est une chemin de longueur 3, de 1 à 4.
— 1, 4, 5 n'est pas un chemin.

1.4 Cycle

Définition 1.4.1 — Cycles. Pour $u \in S$, un cycle est un chemin de longueur supérieure ou égale à 3, de u à u, qui ne passe pas deux fois par le même sommet. Le graphe de la figure 1.3 possède des cycles.

1.5 Connexe

Définition 1.5.1 — Connexe. Un graphe est connexe si $\forall u, v \in S$ il existe un chemin de u à v dans G. Le graphe de la figure 1.3 n'est pas connexe. Le graphe de la figure 1.6 est connexe.

1.6 Arbres

Définition 1.6.1 — Arbre. Un arbre est un graphe connexe sans cycle.

1.7 Arbre couvrants

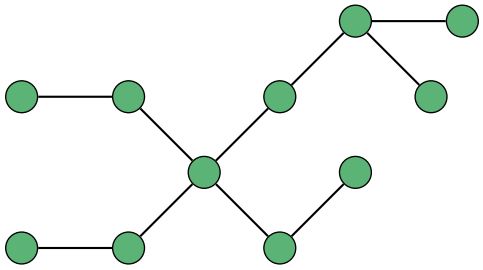


FIGURE 1.6 – Arbre

Définition 1.7.1 — Arbre couvrant. Un arbre $T = (S, A')$ couvre un graphe $G = (S, A)$ si $A' \subseteq A$.

Un graphe à n sommets peut admettre jusqu'à n^{n-1} arbres couvrants.

1.8 Graphes valués

Définition 1.8.1 — Graphe valué. Un graphe est valué (ou pondéré) lorsqu'on y associe une fonction de coût sur les arêtes : $w : S \times S \to \mathbb{R}^+$

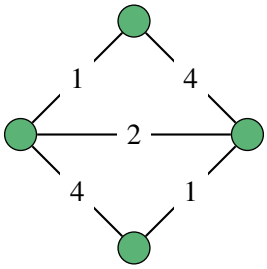


FIGURE 1.7 – Graphe non orienté

1.9 Algorithme de Kruskal (1956)

L'algorithme de Kruskal (1956) permet de trouver un arbre couvrant minimal dans un graphe valué $G = (S, A)$, $w : S \times S \to \mathbb{R}^+$.

1. Trier les arêtes par ordre de coût.
2. Parcourir toutes les arêtes (u, v) dans cet ordre. Si (u, v) ne forme pas de cycle, l'ajouter à l'arbre.

À ne pas confondre

Un **problème** décrit sur chaque donnée quel est le résultat attendu. ex : tri, recherche.

La **structure de données** décrit comment les données sont organisées et comment on y accède.

Un **algorithme** décrit le déroulement des étapes permettant de résoudre un problème.

Un **programme** (ou **implémentation**) est un choix des structures de données, des librairies, du langage, de l'interface...

Exercices

Exercice 1 : Ordres de grandeur

Donner les relations (o, O, θ) entre les fonctions f et g suivantes :

1. $f(n) = 2n$ et $g(n) = 5n + 1$;
2. $f(n) = n^2$ et $g(n) = n^3$;
3. $f(n) = n^2$ et $g(n) = 2n^2 + 3n + 5$;
4. $f(n) = 8(\log n)^2$ et $g(n) = n - 2$;
5. $f(n) = \log(n^2)$ et $g(n) = \log n$;
6. $f(n) = 2^n$ et $g(n) = n^{10}$;
7. $f(n) = 2^{n+1}$ et $g(n) = 2^n$;
8. $f(n) = 2^{2n}$ et $g(n) = 2^n$.

Exercice 2 : Représentation des arbres

Un arbre (pas forcément binaire) peut être représenté de deux façons :

— par les prédécesseurs : chaque nœud contient un pointeur vers son père.
— par liste d'adjacence : chaque nœud contient la liste de tous ses fils.

1. Dessiner un arbre à 8 sommets et donner les deux façons de le représenter.
2. Proposer un algorithme qui transforme un arbre donné par liste d'adjacence en un arbre donné par prédécesseurs. Évaluer sa complexité.
3. Donner un algorithme qui transforme un arbre donné par prédécesseurs en un arbre donné par liste d'adjacence. Évaluer sa complexité.

Solution : `https://cloud.sagemath.com/projects/bb22f23b-6876-4930-9368-3ea2c32cf718/files/Tree.sagews`

2. Cycles et composantes connexes

2.1 Problèmes

Problème 2.1 Soit $G = (S, A)$ et $u, v \in S$.
Répondre $VRAI$ si il existe un chemin de u à v dans G, $FAUX$ sinon.
Problème 2.2 Soit $G = (S, A)$ et $u, v \in S$.
Est-ce qu'ajouter (u, v) à A crée un nouveau cycle ?

Proposition 2.1.1 Problème 2.1 $\Leftrightarrow$ Problème 2.2

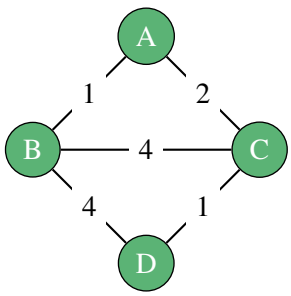
Si il existe un chemin de u à v : $u = u_1, u_2, ...u_k = v$ (qui ne passe pas 2 fois par le même sommet) alors il existe un cycle si on ajoute u, v.

Dans l'autre sens, si ajouter (u, v) crée un nouveau cycle, alors il existe un chemin. En effet, si le morceau cycle passe par $u, v_1, ...v_k, u$ alors G contient le chemin $u, v_k, ..., v_1, v$.

Définition 2.1.1 Une composante connexe d'un graphe $G = (S, A)$ est un sous-ensemble de sommets $S' \subseteq S$ maximal tel que $\forall u, v \in S'$, il existe un chemin de u à v dans G.

Problème 2.3 Soient $G = (S, A)$ et $u, v \in S$, u et v sont-ils dans la même composante connexe ?

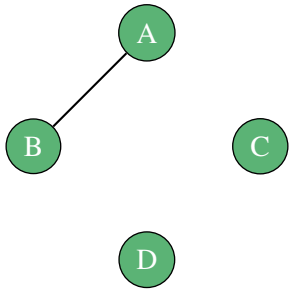
2.2 Application : Kruskal



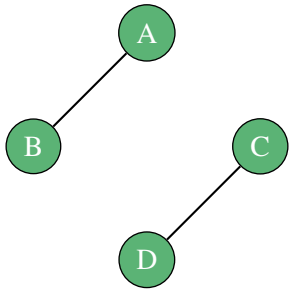
Trier les arêtes : $(A,B)(C,D)(A,C)(B,C)(B,D)$

Ajouter dans l'ordre sans créer de cycle :

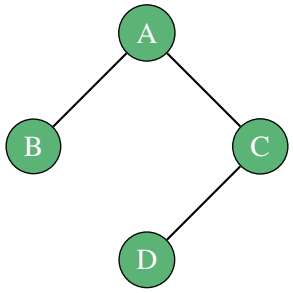
1. Ajouter (A,B), 3 composantes connexes.



2. Ajouter (C,D), 2 composantes connexes.



3. Ajouter (A,C) ? Ok, A et C ne sont pas dans la même composante connexe.



4. Ajouter (B,C). Non.

Objectif : trouver un algorithme performant pour résoudre le problème 2.1.

3. L'algorithme : Union-Find

3.1 Définition

On démarre avec des éléments d'un univers U (ici les sommets d'un graphe). On admet 2 opérations :

— $UNION(u,v)$: associe u et v (ici, on ajoute une arête)
— $FIND(u,v)$: permet de savoir si u et v sont associés, transitive (ici, si il existe un chemin de u à v)

3.2 Exemple

Algorithm 1 Algorithme de Kruskal

```
for (u,v) in A (par ordre croisant d'étiquette) do
  if not FIND(u,v) then
    UNION(u,v)
  else
    end
  end if
end for
```

3.3 Applications

— Segmentation d'image (combien de formes dans une image).
— Construction de labyrinthes.
— Kruskal, arbre couvrant minimal.
— Détection de cycles.

3.4 Idée de l'algorithme

Chaque élément connait son "chef".

On note $Id[u]$ le chef de u.

Pour savoir si u et v sont associés, on note la "hiérarchie" de chef en chef. Si u et v ont le même "PDG", alors u et v sont associés.

Pour associer u et v : $UNION(u,v)$ = le chef du "PDG" de u devient le "PDG" de v.

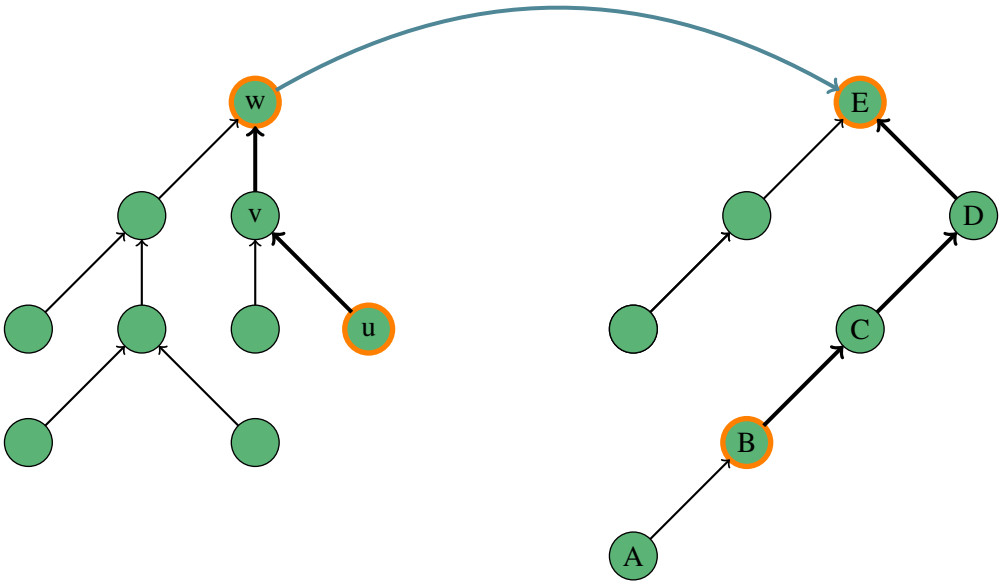


FIGURE 3.1 – $UNION(u,B)$

3.5 Retour sur Kruskal

On teste successivement les arêtes avec $FIND$, tant que la fonction renvoie $FAUX$ on utilise $UNION$, sinon on a fini.

3.6 Implémentation en Python

La classe UF en Python :

```
1  class UF:
2      def __init__(self, U):
3          self.Id = {}
4          for x in U:
5              self.Id[x] = x
6      def root1(self, u):
7          while self.Id[u] != u:
8              u = self.Id[u]
9          return u
10     def find1(self, u, v):
11         return self.root1(u) == self.root1(v)
12     def union1(self, u, v):
13         if not self.find1(u, v):
14             chefu = self.root1(u)
15             chefv = self.root1(v)
16             self.Id[chefu] = chefv
```

3.7 Complexité

— `root1` : O(hauteur de l'arbre)
— `find1` : $2 \times$ hauteur $+ 1$
— `union1` : $4 \times$ hauteur $+ 1$

3.8 *UNION* : Maîtriser la hauteur des arbres

Pour améliorer la complexité on cherche à maîtriser la hauteur des arbres.

Suggestion : on raccroche le plus petit arbre au plus grand ; cela n'augmente pas la hauteur du nouvel arbre sauf si les deux sont de même hauteur.

On maintient un deuxième tableau, qui, pour chaque élément donne sa hauteur.

Au moment de faire l'union, on met à jour `size[u]`, `size[v]`.

```
from UF import UF
class UF2(UF):
    def __init__(self, U):
        super(UF2, self).__init__(U)
        self.size = {}
        for x in U:
            self.size[x] = 0
    def union2(self, u, v):
        chefu = self.root1(u)
        chefv = self.root1(v)
        if self.size[chefu] > self.size[chefv]:
            self.Id[chefv] = chefu
        else:
            self.Id[chefu] = chefv
        if self.size[chefu] == self.size[chefv]:
            self.size[chefv] += 1
```

3.9 *FIND* : Compresser les chemins

Au moment de chercher le représentant d'un élément on compresse le chemin

```
from UF import UF
class UF3(UF):
    def __init__(self, U):
        super(UF2, self).__init__(U)
        self.size = {}
        for x in U: self.size[x] = 0
    def find2(self, u, v):
        return self.root2(u) == self.root2(v)
    def root2(self, u):
        if self.Id[u] != u:
            self.Id[u] = self.root2(Id[u])
        return self.Id[u]
    def union2(self, u, v):
        ru = self.root2(u)
        rv = self.root2(v)
        if self.size[ru] < self.size[rb]:
            self.Id[ru] = self.Id[rv]
        else:
            self.Id[rv] = self.Id[ru]
        if self.size[ru] == self.size[rv]:
            self.size[ru] += 1
```

II Parcours d'arbres

4 Arbres binaires de recherche ABR 21
4.1 Complexité
4.2 Arbres binaires de recherche
4.3 Recherche dans un ABR
4.4 Insertion dans un ABR

4. Arbres binaires de recherche ABR

4.1 Complexité

Pour un algorithme A on note $\text{coût}_A(x)$ le nombre d'opérations élémentaires que l'algorithme effectue sur une entrée x.

Complexité au pire cas : $\text{coût}_A(n) = \max \text{coût}_A(x)$, x de largeur n.

Coût amorti d'un algorithme A : on exécute $A(x_1)A(x_2)\dots A(x_n)$; coût amorti $= \frac{1}{n}\sum_{i=1}^{n} \text{coût}_A(x_i)$

Complexité en moyenne : *"définition mathématique formelle de, à la louche"*, on a une distinction sur les entrées μ, $\text{coût}_A^{\mu}(x) = \sum_{x\sim\mu} \mu(x)\,\text{coût}_A(x)$.

4.2 Arbres binaires de recherche

Définition 4.2.1 — Arbre binaire. Un arbre binaire est soit vide, soit une racine avec un arbre binaire à gauche et un arbre binaire à droite.

En python :

```
1 class ArbreBinaire:
2     # par convention ArbreVide = None
3     def __init__(self, v, g=None, d=None):
4         self.val = v
5         self.gauche = g
6         self.droite = d
```

Définition 4.2.2 — Arbre binaire de recherche. Un arbre binaire de recherche (ABR) est un arbre étiqueté avec les propriétés suivantes :

1. $A.val$ est plus grand que toutes les valeurs de $A.gauche$.
2. $A.val$ est plus petit que toutes les valeurs de $A.droite$.

Définition 4.2.3 — Taille d'un arbre. On écrit $|A|$ la taille de l'arbre A son nombre de sommets.

$$|A| = \begin{cases} 0 \text{ si l'arbre est vide} \\ 1 + |A.gauche| + |A.droit| \text{ sinon} \end{cases}$$

Définition 4.2.4 — Profondeur d'un nœud. On définit la profondeur d'un nœud v dans l'arbre A comme la longueur du chemin de la racine de A jusqu'au sous arbre de racine v :

$$\text{Prof}_A(v) = \begin{cases} 0 \text{ si } A.val = v \\ 1 + \text{Prof}_{A.gauche}(v) \text{ si } A.val > v \\ 1 + \text{Prof}_{A.droite}(v) \text{ si } A.val < v \end{cases}$$

Définition 4.2.5 — Hauteur d'un arbre. On définit la hauteur d'un arbre A :

$$\text{h}(A) = \begin{cases} 0 \text{ si l'arbre est vide} \\ 1 + \max(h(A.gauche), h(A.droit)) \text{ sinon} \end{cases}$$

Remarque : $\text{h}(A) = \max \text{Prof}_A(v), v \in A$

4.3 Recherche dans un ABR

```
1  def TrouverABR(A, v):
2      if v == A.val: return A
3      elif v < A.val and A.gauche is not None:
4          return TrouverABR(A.gauche, v)
5      elif v > A.val and A.droite is not None:
6          return TrouverABR(A.droite, v)
7      return None
```

Proposition 4.3.1 $T(A,v) = \begin{cases} \text{Prof}_A(v) & \text{si } v \in A \\ \text{h}(A) & \text{si } v \notin A \end{cases}$

$\text{coût}_{\text{trouverABR}}(A, v) \leq T(A, v), \forall$ ABR $A, \forall$ valeur $v, |A| \geq 1$

4.3.1 Preuve par récurrence sur $|A|$

Hypothèse d'induction

$H(n) =$ l'énoncé de la proposition 4.3 est vrai $\forall A, v \ |A| \leq n$

Initialisation ("Base")

$n = 1 : A$ a une seule valeur.

si $v \in A$ alors v se trouve à la racine, 1 comparaison.

si $v \notin A$ alors on fera 5 comparaisons.

Hérédité ("Pas d'induction")

Supposons $H(n)$ est vraie pour $n \leq 1$, montrons $H(n+1)$.

Soit A de taille $n+1$.

Si $v \in A$: Si $A.val = v$ on s'arrête après 1 comparaison.

Si $A.val > v$: On fait $3 + T(A.gauche, v)$ comparaisons (au plus)

$T(A.gauche, v) = 5(\text{Prof}_{A.gauche}(v)$ par $H(|A.gauche|)$

On doit montrer que $T(A.gauche, v) + 3 \leq T(A, v)$

$$\begin{aligned} \Leftrightarrow 5(\text{Prof}_{A.gauche}(v) + 1) + 3 \leq T(A, v) &= 5(\text{Prof}_A(v) + 1) \\ \text{D'après la définition de Prof}_A(v), \text{ cas } v < A.val &= 5(\text{Prof}_{A.gauche}(v) + 1 + 1) \\ &= 5(\text{Prof}_{A.gauche}(v) + 1) + 5 \end{aligned}$$

Si $A.val < v$: On fait $T(A.droite, v) + 5$ comparaisons (au plus)

[...]

Si $v \notin A$: Si $A.val = v$, impossible

Si $A.val > v$: $\sharp comparaisons = T(A.gauche, v) + 3$
$\leq 5\mathrm{h}(A.gauche) + 3$ par hypothèse d'induction
$\leq 5(\mathrm{h}(A) - 1) + 3$ par définition de h
$\leq 5\mathrm{h}(A)$

Si $A.val < v$: $\sharp comparaisons = T(A.droit, v) + 5$
$\leq 5\mathrm{h}(A)$

4.4 Insertion dans un ABR

4.4.1 Algorithme

```
1  def InsererABR(A, v):
2      if v == A.val: raise Exception("v deja dans l'arbre")
3      if v < A.val:
4          if A.gauche is not None:
5              InsererABR(A.gauche, v)
6          else: A.gauche = ABR(v, None, None)
7      if v > A.val:
8          if A.droite is not None:
9              InsererABR(A.droite, v)
10         else: A.droite = ABR(v, None, None)
```

On souhaite faire l'analyse amortie de cet algorithme, en supposant qu'on insère les éléments $1, 2, \ldots N$ dans un ordre aléatoire.

coût $_{\text{Amorti}} = \sum_{i=1}^{N}$ coût$_{\text{InsérerABR}}$(la i^{ème} valeur)

On suppose qu'il reste à insérer les valeurs $x_1 < x_2 < \ldots < x_N$ et on choisit d'insérer x_i avec une probabilité de $\frac{1}{N}$.

Si on choisit x_i, on aura $i-1$ éléments à gauche et $n-i$ éléments à droite.

4.4.2 Coût moyen d'insertion de n éléments dans un ABR vide

Proposition 4.4.1 Le coût moyen d'insérer $x_1 < x_2 < \ldots < x_N$ dans un ABR initialement vide, noté C_N est : (où on compte le nombre d'appels récursifs)

$$C_N = \begin{cases} 0 \text{ si } N = 0 \\ 1 \text{ si } N = 1 \\ \sum_{i=1}^{N} \frac{1}{N}[1 + N - 1 + C_{i-1} + C_{N-i}] \text{ sinon} \end{cases}$$

Car : coût(placer x_i) $= 1$
les $N-1$ qui restent passent par la racine,
$x_1 \ldots x_{i-1}$ sont insérés à gauche,
$x_{i+1} \ldots x_N$ sont insérés à droite.

Proposition 4.4.2 $C_N = O(N \log N)$

Démonstration. Prenons $N > 1$

$$\begin{aligned} C_N &= N + \frac{1}{N}\sum_{i=1}^{N}(C_{i-1} + C_{N-i}) \\ &= N + \frac{1}{N}(\sum_{i=1}^{N} C_{i-1} + \sum_{j=1}^{N} C_{N-j}) \\ &= N + \frac{1}{N} 2 \sum_{k=0}^{N-1} C_k \end{aligned}$$

Télescopons :

$$\begin{aligned} NC_N &= N^2 + 2\sum_{k=0}^{N-1} C_k \\ (N-1)C_{N-1} &= (N-1)^2 + 2\sum_{k=0}^{N-2} C_k \\ NC_N - (N-1)C_{N-1} &= N^2 - (N-1)^2 + 2C_{N-1} \\ NC_N &= 2N-1 + (N+1)C_{N-1} \\ \frac{C_N}{N+1} &= \frac{2N-1}{N(N+1)} + \frac{C_{N-1}}{N} \end{aligned}$$

On pose : $t(N) = \frac{2N-1}{N(N+1)} \simeq \frac{2}{N}$

$$\begin{aligned} \frac{C_N}{N+1} &= t(N) + \frac{C_{N-1}}{N} \\ \frac{C_N}{N+1} &= t(N) + t(N-1) + \frac{C_{N-2}}{N-1} \\ &= t(N) + t(N-1) + t(N-2) + \frac{C_{N-3}}{N-2} \\ &= \sum_{i=2}^{N} t(i) + \frac{1}{2} \\ &\approx \sum_{i=1}^{N} \frac{2}{i} + \frac{1}{2} \\ &\approx 2\sum_{i=1}^{N} \frac{1}{i} \end{aligned}$$

$\frac{C_N}{N+1} \approx 2 * N \ln n$

On conclut que $C_N = 2(N+1)N\ln(N)$ et que le coût amorti $\frac{C_N}{N} = 2(N+1)\ln(N)$

■

III Parcours de graphes

5 Parcours d'arbres k-aire 27
5.1 Structure de donnée utilisée
5.2 Parcours en largeur des arbres
5.3 Propriétés du parcours en largeur
5.4 Parcours en profondeur des arbres
5.5 Propriétés du parcours en profondeur des arbres

6 Parcours de graphes 31
6.1 Représentation des graphes (non orientés)
6.2 Parcours en largeur des graphes
6.3 Parcours en profondeur des graphes

5. Parcours d'arbres k-aire

On appelle arbre k-aire un arbre dont chaque nœud a au plus k fils.

Problème

Étant donné un arbre/graphe, on veut parcourir chaque sommet une et une seule fois.

5.1 Structure de donnée utilisée

Une file Q, *FIFO (First In First Out)*, avec les opérations suivantes :

— $Q.push(v)$: ajouter v à la fin de la file.
— $u = Q.pop()$: retire de le premier élément de la liste et le place dans la variable u.
— $Q.vide()$: retourne Vrai si la file est Vide, Faux sinon.

On ajoute les informations suivantes aux sommets au moment du parcours, **pour chaque sommet** A :

— Datage : $A.debut$, $A.fin$ les instants où on découvre/quitte A
— Couleur :
  — blanc : pas encore visité
  — gris : en cours
  — noir : terminé
— Distance : pour le parcours en largeur, la distance à la racine

```
1 class Noeud:
2     def __init__(self, val, fils=[]):
3         self.val = v
4         self.fils = fils
5         self.couleur = "blanc"
```

On traite les arbres à arité (nombre d'enfants) variable. Chaque sommet A peut avoir un nombre arbitraire de fils donnés dans une liste $A.fils$.

5.2 Parcours en largeur des arbres

```
def BFS(A):
    t = 0 ; d = 0
    Q = [(A,0)] # Queue
    A.couleur = "gris";
    A.debut = t; t += 1
    while not(Q.vide()):
        noeud, d = Q.pop(0) # pop first
        noeud.couleur = "noir"
        noeud.fin = t
        t += 1
        for v in noeud.fils:
            v.couleur = "gris"
            v.debut = t; t += 1
            Q.append((v, d + 1)) # push last
```

5.3 Propriétés du parcours en largeur

Proposition 5.3.1 Les sommets sont visités par ordre de profondeur.

On découpe en deux étapes la preuve de la proposition 5.3.1.

Proposition 5.3.2 Supposons que la file contienne les valeurs $(v_1, d_1), \dots, (v_k, d_k)$, lorsque (v_i, d_i) entre dans la file $d_i = Prof_A(v_i)$.

Preuve de la proposition 5.3.2

Par induction sur t.

— Au temps $t = 0$: La file contient $(A, 0)$ et par définition $Prof_A(t) = 0$.

— Au temps $t+1$: Supposons que Q contienne $(v_1, d_1), \dots, (v_k, d_k)$ où $d_i = Prof_A(v_i)$. On retire (v_1, d_1) de la file. On ajoute $(f_1, d_1+1), \dots, (f_e, d_1+1)$ où les f_i sont les fils de v_1. Les f_i sont bien à hauteur $d_1 + 1$ dans l'arbre.

Proposition 5.3.3 Si au temps t, Q contient $(v_1, d_1), \dots, (v_k, d_k)$ alors $d_1 \leq d_2 \leq \dots \leq d_k$ et $d_k - d_1 \leq 1$.

Preuve de proposition 5.3.3

Par induction sur t.

— Au temps $t = 0$: La file contient $(A, 0)$, la proposition est donc vraie.

— Supposons qu'à l'instant t, Q contienne $(v_1, d_1), \dots, (v_k, d_k)$, par hypothèse d'induction, ceux-ci vérifient la proposition. On retire (v_1, d_1) et on ajoute ses fils $(f_1, d_1+1), \dots, (f_e, d_1+1)$ de v_1 à la suite.

Vérification : 2 cas possibles $d_1 = d_k$ ou $d_1 + 1 = d_k$ (par hypothèse d'induction). On a $(v_2, d_2), \dots, (v_k, d_k)$ et $(f_1, d_1+1), \dots, (f_e, d_1+1)$.

1. Les distances sont encore triées, elles sont triées jusqu'à d_k, puis pour $f_1, \dots, f_e$:
   — $d_k = d_1$, on ajoute $d_1, \dots, d_1$.
   — $d_k = d_1 + 1$, on ajoute $d_1 + 1, \dots, d_1 + 1$.
2. $d_1 + 1 - d_2 < 1$? Par hypothèse, $d_1 \leq d_2 \leq d_1 + 1$, ok.

Proposition 5.3.4 Chaque sommet entre et sort de la file une et une seule fois.

Preuve de la proposition 5.3.4

1. Chaque sommet qui entre dans la file doit en sortir : l'algorithme ne se termine que lorsque la file est vide.

2. Un sommet qui est entré dans la file ne peut pas y ré-entrer. En effet, il ne rentre que quand son père sort. Mais par récurrence son père ne peut pas ré-entrer dans la file.
3. Chaque sommet de l'arbre entre dans la file. Sinon, son père n'est pas entré, ainsi de suite tous ses ascendants ne sont pas rentrés jusqu'à la racine.

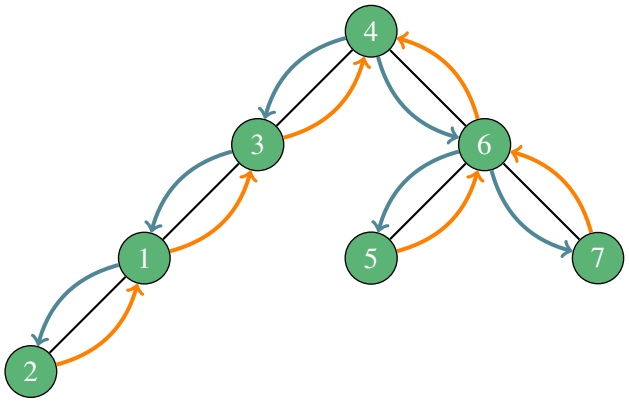
Conclusion : Le parcours en largeur parcourt les sommets une et une seule fois par ordre croissant de profondeur. La complexité est $2|A|$ (en nombre de $pop()$ et $push(v)$).

5.4 Parcours en profondeur des arbres

Parcours en profondeur des arbres à arité variable :

```
1 def DFS(A, t = 0):
2     A.couleur = "gris"; A.debut = t; t += 1
3     for v in A.fils:
4         t = DFS(A, t)
5     A.couleur = "noir"; A.fin = t; t += 1
6     return t
```

Exemple



Remarque : si on associe au parcours en profondeur "je vais visiter i" $\mapsto \underset{i}{(}$ et "je quitte i" $\mapsto \underset{i}{)}$ on obtient une expression correctement parenthésée ; ici : $\underset{4}{(}\ \underset{3}{(}\ \underset{1}{(}\ \underset{2}{(}\ \underset{2}{)}\ \underset{1}{)}\ \underset{3}{(}\ \underset{6}{(}\ \underset{5}{)}\ \underset{5}{(}\ \underset{7}{)}\ \underset{7}{)}\ \underset{6}{)}\ \underset{4}{)}$

5.5 Propriétés du parcours en profondeur des arbres

Proposition 5.5.1 Le parcours en profondeur a les propriétés suivantes :
- $\forall A\ A.debut < A.fin$
- Avant $A.debut$, $A.couleur =$ "blanc"
- Entre $A.debut$ et $A.fin$, $A.couleur =$ "gris"
- Après $A.fin$, $A.couleur =$ "noir"

Proposition 5.5.2 Pour toute paire de sommets A, A' les intervalles $I_A = (A.debut, A.fin)$ et $I_{A'} = (A'.debut, A'.fin)$ ne peuvent se chevaucher.

Preuve de la proposition 5.5.2

Supposons $A.debut < A'.debut < A.fin$, alors entre $A.debut$ et $A.fin$, les appels récursifs, y compris à $DFS(A)$ se sont terminés donc $A'.debut < A'.fin < A.fin$.

Proposition 5.5.3 A' est descendant de $A \Leftrightarrow A.debut < A'.debut < A'.fin < A.fin$

Proposition 5.5.4 L'ensemble des sommets gris forment un chemin de la racine jusqu'à un sommet de l'arbre.

Preuve de la proposition 5.5.4 Soient $v_1, \ldots, v_k$ les sommets gris à l'instant t. Supposons $v_1.debut < v_2.debut < \ldots < v_1.fin$

On a montré : $v_1.debut < v_2.debut < \ldots < v_1.fin$

D'après la proposition 5.5.2, on a que v_{i+1} descendant de v_i $\forall i$ $1 \leq i \leq k$. Mais il faut monter que v_{i+1} est fils de v_i. [...]

6. Parcours de graphes

6.1 Représentation des graphes (non orientés)

On ajoute les informations suivantes **pour chaque sommet** S :

- $S.debut, S.fin$: pour le datage, les instants où on découvre/quitte S
- $S.couleur$:
  - blanc : pas encore visité
  - gris : en cours
  - noir : terminé
- $S.dist$: pour le parcours en largeur, la distance à la racine
- $S.voisins$: la liste des voisins.
- $S.pred$: le prédécesseur de S dans le parcours.

```
1 class Noeud:
2     def __init__(self, val, voisins=[]):
3         self.val = val
4         self.voisins = voisins
5         self.couleur = "blanc"
6         self.dist = 0
```

6.2 Parcours en largeur des graphes

Problème 1 : si on applique l'algorithme des arbres, on obtient une boucle infinie.
Problème 2 : certains sommets ne seront jamais visités.
Supposons à partir de maintenant que G est connexe.

Définition 6.2.1 — Chemins. Notations :
$u \to_G^d v$: il existe un chemin de longueur d de u à v dans G.
$u \to_G^* v$: il existe un chemin de u à v dans G.
$\delta_G(u,v)$: $\min\{d : u \to_G^d v\}$
$V_k(s)$: $\{v \in S : \delta_G(u,v) = k\}$

```
def BFS(S):
    t = 0 ; S.dist = 0; s.pred = None; s.couleur = "gris"
    Q = [S] # Queue
    S.debut = t; t += 1;
    while not(Q.vide()):
        noeud = Q.pop(0) # pop first
        noeud.couleur = "noir"
        noeud.fin = t; t += 1
        for v in noeud.voisins:
            if v.couleur == "blanc":
                v.pred = noeud
                v.dist = noeud.dist + 1
                v.debut = t; t += 1
                v.couleur = "gris"
                Q.push(v) #push last
```

Proposition 6.2.1 À l'appel de $BFS(s)$:

1. Les sommets (accessibles depuis s) sont parcourus par ordre de distance à s.
2. Après le parcours d'un sommet u, $u.dist$ est égal à la distance de u à s.
3. Si le sommet u est à distance d de s, alors il existe un chemin de s à u de longueur d dont la dernière arête est $(u.pred, u)$.

Preuve de la proposition 6.2.1.1

Il faut se convaincre qu'à l'instant où on traite le dernier sommet à distance d de s, tous les sommets à distance $d+1$ soient dans la file.

On a montré pour les arbres que la file ne contient que des sommets à distance $d+1$; il faut montrer que tous sont dans la file.

Posons $u \in V_{d+1}$ quelconque.

Comme $\delta(s,u) = d+1$, $\exists v$, $\delta(s,u) = d$ et $(u,v) \in A$.

Par induction, v a été traité avec les sommets de distance d et donc u :

— s'il n'avait pas déjà été visité au moment de traiter v, a été ajouté à la file

— s'il avait déjà été visité, u a déjà été mis dans la file. Il a dû être mis dans la file pour un autre sommet v'.

$\delta(s,v')$ est nécessairement égal à d sinon $\delta(s,u) = \delta(s,v') + 1 < d+1$.

Proposition 6.2.2 Tous les sommets accessibles à partir de s sont visités une et une seule fois.

Définition 6.2.2 — Graphe des prédécesseurs. Après le parcours (BFS ou DFS) à partir de $s \in S$ d'un graphe $G = (S, A)$, on appelle graphe des prédécesseurs : $G_{pred} = (S, A')$ avec $A' = \{(u, u.pred),\ u \in S \setminus \{S\}\}$

Proposition 6.2.3 Si G est connexe, G_{pred} est un arbre ; on l'appelle arbre de parcours.

Preuve de la proposition 6.2.3

G_{pred} a $n = |S|$ sommets et $n-1$ arêtes ; G_{pred} est connexe car on a parcouru tous les sommets en passant par des arêtes du graphe.

Proposition 6.2.4 La complexité de $BFS(S)$ en nombre de comparaisons est : $\sum_{x \in S} \sum_{v \in x} 1 = 2|A|$ (car $\sum_{x \in S}$ représente la boucle `while`, 1 fois pour chaque sommet retiré de la file ; $\sum_{v \in x}$ la boucle `for`)

6.3 Parcours en profondeur des graphes

6.3.1 Version récursive

```
def DFS(S, t = 0):
    S.couleur = "gris"; S.debut = t; t += 1
    for v in A.fils:
      if v.couleur == "blanc":
         v.pred = S
           t = DFS(v, t)
    S.couleur = "noir"; S.fin = t; t += 1
    return t
```

6.3.2 Version itérative

```
def DFS(S):
   pile = [S]; S.couleur = "gris"
   while not len(pile) == 0:
      next = pile[-1] # head
      for v in next.voisins:
         if v.couleur == "blanc":
            v.couleur = "gris"
            pile.push(v)
            break
      else:
         next.couleur = "noir"; pile.pop(-1)
```

Exemple

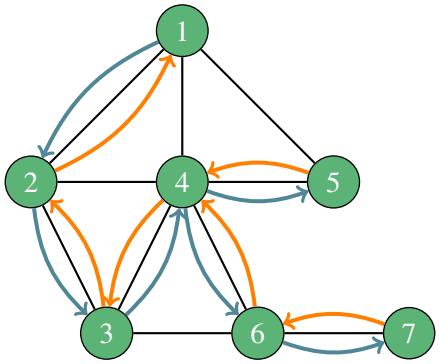


FIGURE 6.1 – G

Remarque : si on associe au parcours en profondeur "je vais visiter i" $\mapsto$ $\underset{i}{(}$ et "je quitte i" $\mapsto$ $\underset{i}{)}$ on obtient une expression correctement parenthésée ; ici : $\underset{1}{(}\,\underset{2}{(}\,\underset{3}{(}\,\underset{4}{(}\,\underset{6}{(}\,\underset{7}{(}\,\underset{7}{)}\,\underset{6}{)}\,\underset{5}{(}\,\underset{5}{)}\,\underset{4}{)}\,\underset{3}{(}\,\underset{2}{)}\,\underset{1}{)}$

Définition 6.3.1 — Chemins. On écrit :
- $u \to_G^* v$: il existe un chemin de u à v dans G.
- $u \Rightarrow_G^* v$: il existe un chemin orienté de u à v dans $G_{pred} = \{(s.pred, s),\ s \in S\}$.

Théorème 6.3.1 — Chemin blanc. Si $u \Rightarrow_{G_{pred}}^* v \Leftrightarrow$ au temps $t = u.debut$, un chemin blanc relie u à v dans G.

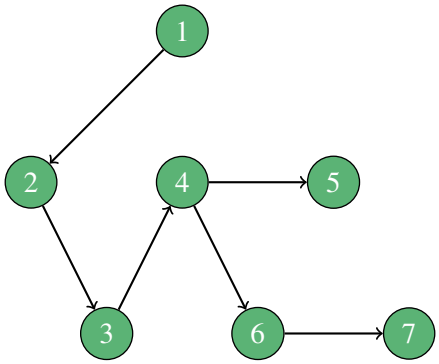


FIGURE 6.2 – G_{pred}

Proposition 6.3.2 Le parcours en profondeur a les propriétés suivantes : $\forall$ sommet $u \in S$

1. $u.debut < u.fin$
2. $\forall t < u.debut \;\; u.couleur =$ blanc
3. $\forall t : u.debut \leq t \leq u.fin \;\; u.couleur =$ gris
4. $\forall t > u.fin \;\; u.couleur =$ noir

Proposition 6.3.3 Pour toute paire de sommets u, v les intervalles $I_u = (u.debut, u.fin)$ et $I_v = (v.debut, v.fin)$ ne peuvent se chevaucher.

Proposition 6.3.4 Pour toute paire de sommets $(u\,v) \;\; u \Rightarrow^*_G v \Leftrightarrow u.debut < v.debut < v.fin < u.fin$

Preuve du théorème du "chemin blanc"

($\Rightarrow$) Supposons que $u \Rightarrow^*_{G_{pred}} v$ par le chemin $u = u_0 \Rightarrow^1_{G_{pred}} u_1 \Rightarrow^1_{G_{pred}} \ldots \Rightarrow^1_{G_{pred}} u_k = v$.
D'après la proposition 6.3.4 les intervalles s'emboîtent : $\underset{u_0}{(}\;\underset{u_1}{(}\;\underset{u_2}{(}\;\ldots\;\underset{u_k}{(}\;\underset{u_k}{)}\;\ldots\;\underset{u_2}{)}\;\underset{u_1}{)}\;\underset{u_0}{)}$
D'après la proposition 6.3.2 : au temps $t = u.debut \;\; u_1, u_2, \ldots u_k$ sont blancs.
Donc le chemin $u = u_0 \Rightarrow^1_{G_{pred}} u_1 \Rightarrow^1_{G_{pred}} \ldots \Rightarrow^1_{G_{pred}} u_k = v$ est blanc.

($\Leftarrow$) Supposons qu'il existe un chemin blanc de u à v dans G au temps $t = u.debut$
$u = u_0 \Rightarrow^1_{G_{pred}} u_1 \Rightarrow^1_{G_{pred}} \ldots \Rightarrow^1_{G_{pred}} u_k \Rightarrow \ldots \Rightarrow u_{i-1} \rightarrow u_i \rightarrow \ldots u_k = v$
Soit i le point le plus petit tel que $(u_{i-1}, u_i) \notin G_{pred}$.
On va montrer qu'on peut trouver un chemin dans G_{pred}.
Posons $k < i$ le plus petit tel que $(u_k, u_i) \in G$.
Examinons la structure au moment $t' = u_k.fin$.
On veut montrer qu'on peut aller jusqu'à i avec un chemin dans G_{pred} .
Au temps t' , quelle est la couleur de u_i ?

- $u_i =$ blanc ? Non, on visite tous les voisins blancs de u_k avant $u_k.fin$
- $u_i =$ gris ? On avait $u_0 \Rightarrow^* u_i \Rightarrow u_k$ or $k < i$, donc impossible
- $u_i =$ noir ? 2 possibilités :
  - Soit $\underset{u_0}{(}\;\underset{u_i}{(}\;\underset{u_i}{)}\;\underset{u_k}{(}\;\underset{u_k}{)}\;\underset{u_0}{)}$ d'après la proposition 6.3.3
  - Soit $\underset{u_0}{(}\;\underset{u_k}{(}\;\underset{u_i}{(}\;\underset{u_i}{)}\;\underset{u_k}{)}\;\underset{u_0}{)}$ d'après la proposition 6.3.4.

  Dans les deux cas $u_0 \Rightarrow^*_{G_{pred}} u_i$

Applications des parcours de graphes

— Recherche de cycles (deadlock, BDD)
— Plus courts chemins dans les graphes
— Détection de composante connexe (reconnaissance de motifs dans une image)
— Tri topologique (gestion de projet)
— Recherche de stratégie optimale dans les jeux.

IV Algorithmes de graphes

7 Algorithme de Kruskal 37
7.1 Implémentation de l'algorithme de Krukal
7.2 Correction de l'algorithme de Kruskal

8 Algorithme de Prim 39
8.1 Implémentation de l'algorithme de Prim (version optimisée)
8.2 Complexité de l'algorithme de Prim

9 Les plus cours chemins : Dijkstra 41
9.1 Algorithme de Dijkstra
9.2 Preuve de l'algorithme de Dijkstra

10 Algorithme de Bellman-Ford 45
10.1 Idée
10.2 Implémentation en Python
10.3 Preuve

11 All pairs shortest paths 47
11.1 Dijkstra, plusieurs fois
11.2 Modification de Bellman-Ford
11.3 En doublant la longueur des chemins à chaque itération
11.4 Algorithme de Floyd-Warshall

12 Cycles négatifs 51
12.1 Algorithme pour détecter les cycles négatifs

Index 53

7. Algorithme de Kruskal

Définition 7.0.1 — Graphe valué. Un graphe $G = (S, A)$ est valué (ou pondéré) si on y associe une fonction de coût sur ses arêtes : $w : A \longrightarrow \mathbb{R}^+$.

Définition 7.0.2 — Arbre couvrant minimal. Soit $G = (S, A)$ et w une fonction de coût, trouver $A' \subseteq A$ tel que $T = (S, A')$ soit un arbre couvrant de cout minimal $\text{coût}(T) = \sum_{(u,v)\in A'} w(u, v)$

7.1 Implémentation de l'algorithme de Krukal

```
1  from UF import UF
2  def Kruskal (G, w):
3      T = []; cout = 0;
4      A = G.A # l'ensemble des arretes de G
5      S = sorted(A) # A trie selon w croissant
6      for (u, v) in S:
7          if not UF.find(u, v): # si (u, v) ne forme pas de cycle
8              T.append((u, v))
9              cout += w[(u, v)]
10             UF.union(u, v)
11     return (T, cout)
```

Complexité : $m \log m$ avec $m =$ nombre d'arêtes

7.2 Correction de l'algorithme de Kruskal

Définition 7.2.1 — Arbres couvrants minimaux. $ACM(G, w) =$ l'ensemble des arbres couvrants minimaux de $G = (S, A) : T' = (S, A')$ avec $A' \subseteq A$.

Définition 7.2.2 — Compatibilité. Pour toute solution partielle $T' \subseteq G$, on dit que T' est **compatible** avec $ACM(G, w)$ et on note $T' \sqsubseteq ACM(G, w)$ si $\exists T \in ACM(G, w)$ tel que $T' \subseteq T$.

7.2.1 Lemme d'optimalité

Soit G connexe, $G = (S, A)$ et $w : A \longrightarrow \mathbb{R}^+$. Si $T' \sqsubseteq ACM(G, w)$, $T' = (S, A')$ et $S = S_1 \cup S_2$ $S_1 \cap S_2 = \varnothing$ tels que $A' \cap (S_1 \times S_2) = \varnothing$, alors on peut ajouter $(u, v) \in S_1 \times S_2$, de coût minimal parmi les arêtes $S_1 \times S_2 \cap A$. Autrement dit, $T \cup \{(u, v)\} \sqsubseteq ACM(G, w)$

Preuve

On a $G = (S, A)$ connexe, $w \longleftarrow \mathbb{R}^+$ $S = S_1 \cup S_2$ $S_1 \cap S_2 = \varnothing$
$T' \subseteq T \in ACM(G, w)$
$T' \cap S_1 \times S_2 = \varnothing$
Soit (u, v) **de moindre coût** parmi $A \cap S_1 \times S_2$
On veut montrer que $T' \cup (u, v) \sqsubseteq ACM(G, w)$

— **cas 1** : si $(u, v) \in T$ alors $T' \cup (u, v) \subseteq ACM(G, w)$
— **cas 2** : si $(u, v) \notin T$, comme G est connexe, T est un ACM.
T est connexe, $(u', v') \in S_1 \times S_2 \cap T$ car il existe un chemin de u à v dans T.
On a pris (u, v) **de poids minimal** ; donc $w(u, v) \leq (u', v')$.
Regardons $\widetilde{T} = T \setminus \{(u', v')\} \cup \{(u, v)\}$

1. coût de T : coût dans S_1 + coût dans S_2 + $w(u', v')$
   coût de $\widetilde{T}$: coût dans S_1 + coût dans S_2 + $w((u, v))$ or $(u, v) \leq w(u', v')$
   Donc coût de $\widetilde{T}$ < coût de T
2. Si T est un arbre couvrant, $\widetilde{T}$ l'est aussi.
   Donc $\widetilde{T} \in ACM(G, w)$ et $T' \cup (u, v) \subseteq ACM(G, w)$

Dans le cas de l'algorithme de Kruskal, supposons qu'à une itération donnée, on ait choisi les arêtes T' et on a montré $T \sqsubseteq ACM(G, w)$.

Posons S_1 la composante connexe qui contient u et $S_2 = S \setminus S_1$.

(u, v) que choisit Kruskal est celle de poids minimal parmi les arêtes qui restent qui ne forme pas de cycle, comme $u \in S_1$, forcément $v \notin S_1$. Donc on peut l'ajouter d'après le lemme d'optimalité.

8. Algorithme de Prim

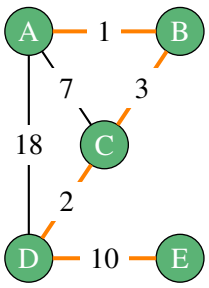
On fait pousser un arbre T.
À chaque instant T est un arbre.
On ajoute l'arête de coût minimal parmi les arêtes qui sortent de l'arbre.
Pour trouver la meilleure on utilise une file de propriété (un tas min).

8.1 Implémentation de l'algorithme de Prim (version optimisée)

On met les sommets dans le tas et on note le prédécesseur du sommet (arête) qui donne la meilleure priorité. En cours d'algorithme la priorité des sommets dans le tas peut changer.

```
1  from FilePriorite import FilePriorite
2  def Prim(G, w, s):
3      inf = float("inf")
4      H = FilePriorite()
5      T = []
6      s.priorite = 0;   H.inserer(0, s)
7      for u in G.sommets:
8          u.pred = None
9          if u != s :
10             H.inserer(inf, u)
11             u.priorite = inf
12     while not H.is_empty():
13         p, u = H.pop_min()
14         if u.priorite != inf:
15             if u.pred is not None: T.append((u.pred, u))
16             for v in u.voisins:
17                 if v.idx is not None: #v n'est pas sorti du tas
18                     if w[(u, v)] < v.priorite:
19                         v.priorite = w[(u, v)]
20                         H.diminuer_cle(v.idx, v.priorite)
21                         v.pred = u
22     return T
```

Exemple



Tas :	$(A,0)$	(B,∞)	(C,∞)	(D,∞)	(E,∞)
	$(A,0)$	(B,∞)	(C,∞)	(D,∞)	(E,∞)
		$(B,1)$	$(C,7)$	$(D,18)$	(E,∞)
			$(C,3)$	$(D,18)$	(E,∞)
				$(D,3)$	(E,∞)
					$(E,10)$

8.2 Complexité de l'algorithme de Prim

Initialisation : $n\times$ insérer $= O(n\log n)$
Boucle while : n fois
Corps de la boucle : $pop + for \times update = \log n + deg(u) \times \log n \approx n\log n + 2m\log n$

9. Les plus cours chemins : Dijkstra

Communisme vs Capitalisme

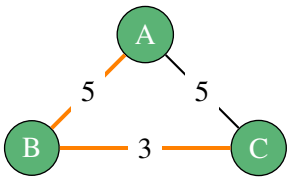


FIGURE 9.1 – Arbre Couvrant minimal, coût = 8

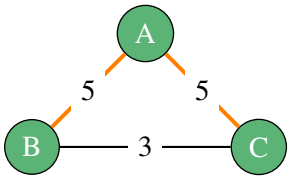


FIGURE 9.2 – Plus courts chemins depuis A

Étant donné un graphe valué G, w et un sommet de départ s. On veut calculer les plus courts chemins de s à v dans G $\forall v \in S$.

Définition 9.0.1 — Notations. Si $C = (s = u_0, u_1, u_2, \ldots u_k = v)$ est un chemin de s à v dans G, on écrit :

- $s \xrightarrow{C}_G v$
- $w(C) = \sum_{i=1}^{k} w(u_{i-1}, u_i)$
- $\delta(s, v) = min\{w(C) : s \xrightarrow{C}_G v\}$

Proposition 9.0.1 On peut toujours trouver des chemins $C_u s \xrightarrow{C} u$ tel que l'union des C_u forme un arbre.

Preuve Supposons qu'on ait 2 sommets u, v des plus courts chemins C_u, C_v dont l'union forme un cycle.

Les segments C_1, C_2 $x \xrightarrow{C_1} y$ $x \xrightarrow{C_2} y$ ont le même coût, sinon supposons $w(C_1) < w(C_2)$ alors il y a un chemin plus court de s à v qui passe par C_1 ; C_2 n'est pas un plus court chemin, impossible.

9.1 Algorithme de Dijkstra

On modifie l'algorithme de Prim, la notion de priorité change, maintenant c'est la distance à s.

améliorer la priorité ≡ améliorer la distance à s

```
from FilePriorite import FilePriorite
def Dijkstra(G, w, s):
    inf = float("inf")
    H = FilePriorite()
    T = []
    s.priorite = 0;    H.inserer(0, s)
    for u in G.sommets:
        u.pred = None
        if u != s :
            H.inserer(inf, u)
            u.priorite = inf
    while not H.is_empty():
        p, u = H.pop_min()
        if u.priorite != inf:
            if u.pred is not None: T.append((u.pred, u))
            for v in u.voisins:
                if v.idx is not None: #v n'est pas sorti du tas
                    if u.priorite + w[(u, v)] < v.priorite:
                        v.priorite = u.priorite + w[(u, v)]
                        H.diminuer_cle(v.idx, v.priorite)
                        v.pred = u
    return T
```

9.2 Preuve de l'algorithme de Dijkstra

Définition 9.2.1 Notons $u.prio^t$ la valeur de $u.prio$ à la $t^{\text{ième}}$ itération.

Théorème 9.2.1 À l'itération t, si $u \notin tas$ alors $u.prio^t = \delta(s,u)$.

On utilise sans démontrer trois propriétés :

(P1) S'il n'y a pas de chemin de s à u alors $u.prio^t = \infty \ \forall t$

(P2) "Borne supérieure" : $\forall u, \ \forall\ u.prio^t \geq \delta(s,u)$ et $u.prio^t \geq u.prio^{t+1}$

(P3) Si $s \longrightarrow_G^* u \longrightarrow^1 v$ est un plus court chemin de s à v et $u.prio^t = \delta(s,u)$, alors $\forall t' > t \ v.prio^{t'} = \delta(s,v)$

Preuve du théorème

Supposons qu'à l'itération t, u sorte du tas.

Regardons T^t l'ensemble des arêtes déjà choisies.

Soit C un plus court chemin de s à u, et x, y la première arête de C telle que $(x,y) \notin T^t$.

$s \xrightarrow{C_1 \subseteq T} x \xrightarrow{\notin T} {}^1\ y \xrightarrow{C_2} u$

On va montrer que $y = u$.

(I) Montrons que : $y.prio^t = \delta(s,y)$

(*) On utilise le fait que sur un plus court chemin de u_0 à u_k les segments initiaux de u_0 à u_i (avec $0 \leq i \leq k$) sont aussi des plus courts chemins

On a que $x \in T^t$, supposons qu'il a été ajouté à l'instant $t' < t$.

Par hypothèse d'induction : $x.prio^{t'} = \delta(s,x)$

À l'itération t', y est encore dans le tas et y est voisin de x, et on aura posé $y.prio^{t'} \leq x.prio^{t'} + w(x,y)$ or $s \longrightarrow^i x \longrightarrow^1 y$ est un plus court chemin (par (*)).

$y.prio^{t'} \leq \delta(s,x) + w(x,y) =$ longueur du chemin $= \delta(s,y)$ et par P2 $y.prio^t = \delta(s,y)$

(II) On montre que $C_2 = \varnothing$

Donc on conclut que $y = u$, et par (I) $u.prio^t = y.prio^t = \delta(s,y) = \delta(s,u)$

À l'itération t, $y \in T^t$, mais y a une priorité $\neq \infty$ donc y est dans le tas, tout comme u qui a été choisi.

$$\begin{array}{rcll} u.prio^t & \leq & y.prio^t & \text{car } u \text{ sort du tas} \\ & = & \delta(s,y) & \text{(II)} \\ & \leq & \delta(s,u) & \text{car } y \text{ précède } u \text{ sur un plus court chemin} \\ & \leq & u.prio^t & \text{(P2)} \end{array}$$

$\Rightarrow$ toutes ces quantités sont égales $\Rightarrow y = u$

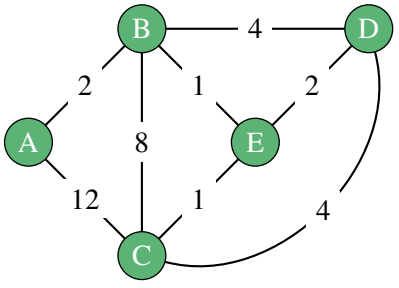
10. Algorithme de Bellman-Ford

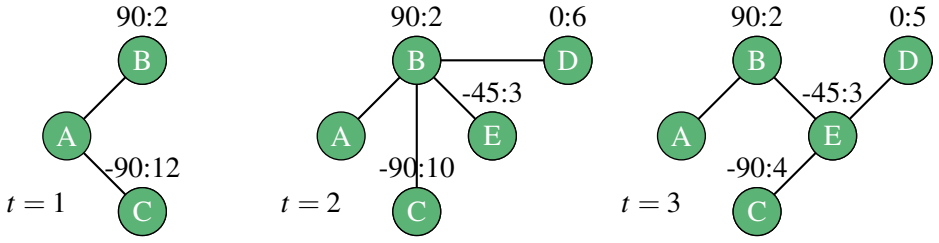
Cet algorithme permet de trouver les plus courts chemins depuis un sommet source donné. Il est cependant moins performant que Dijkstra, mais est distribuable, chaque sommet du réseau connaît ses voisins, propage les résultats des calculs intermédiaires à ses voisins.

10.1 Idée

Commencer avec $d(s) = 0$ et $d(u) = \infty \; \forall u \neq s$.
À chaque itération, on parcourt toutes les arêtes du graphe ; $(u, v) \in A$: si $d(u) + w(u, v) < d(v)$
On pose $d(v) = d(u) + w(u, v)$ et $pred(v) = u$.

Exemple





On remarque qu'à l'itération t on trouve des plus courts chemins qui parcourent au plus t arêtes.

Un plus court chemin de s à u parcourt au plus $n-1$ arêtes. Pour que l'algorithme soit correct, il suffit de montrer qu'à l'itération t on a calculé les plus courts chemins passant par au plus t arêtes et répéter $n-1$ fois pour avoir tous les plus courts chemins de s aux autres sommets.

10.2 Implémentation en Python

```
1  def Bellman_Ford(G, w, s):
2      inf = float("inf")
3      for u in G.sommets:
4          if u != s: u.dist = inf
5          u.pred = None
6      s.dist = 0
7      for t in range(1, len(G.sommets)):
8          for u in G.sommets:
9              for v in u.voisins:
10                 if v.dist > u.dist + w[(u, v)]:
11                     v.dist = u.dist + w[(u, v)]
12                     v.pred = u
13     return [(u.pred, u) for u in G.sommets if u != s]
```

Complexité : $n \times m$

10.3 Preuve

Définition 10.3.1 On note :
- $\delta^t(u,v)$ = longueur du plus court cheminde u à v en passant par au plus t arêtes.
- $v.dist^t$ = valeur de $v.dist$ à l'itération t.

Proposition 10.3.1 $\forall v \in S\ d^t(v) = \delta^t(s,v)$

Démonstration. Par récurrence sur t.

Lorsque $t = 0$, un seul chemin de longueur 0 de s à s.

À l'itération $t > 0$, par hypothèse d'induction, $\forall x\ x.dist^{t-1} = \delta^{t-1}(s,x)$

Soit C_v un plus court chemin de s à v parcourant au plus t arêtes. Si C_v a moins de t arêtes alors on l'a découvert à une itération précédente $t' < t$ donc $v.dist^{t'} = \delta^{t-1}(s,v)$. Si C_v a t arêtes, le chemin $s \longrightarrow x$ est un plus court chemin qui parcourt au plus $t-1$ arêtes (sinon on trouverait un meilleur chemin de t arêtes vers v).

On peut appliquer l'hypothèse d'induction $x\ x.dist^{t-1} = \delta^{t-1}(s,x)$.

À l'itération t on considère x et son voisin v et on posera :

$$\begin{aligned} v.dist &\leq x.dist + w(x,v) \\ &= \delta^{t-1}(s,x) + w(x,v) \\ &= \delta^t(s,v) \end{aligned}$$

■

11. All pairs shortest paths

"Plus courts chemins toutes sources toutes destinations"

Étant donné un graphe G et une fonction de coût w, on veut trouver $\forall u, v \in S$ un plus court chemin de u à v.

11.1 Dijkstra, plusieurs fois

Lancer Dijkstra sur tous les sommets ; complexité en temps : $n \times (m + n \log n)$ (n sommets, m arêtes).

11.2 Modification de Bellman-Ford

Idée : Au temps t, calculer $\delta^t(u\,v)$ en utilisant $\delta^{t-1}(u\,v)$ $\forall u, v$.

Initialement $\delta^1(u\,v) = w(u, v)$ $\forall u, v$

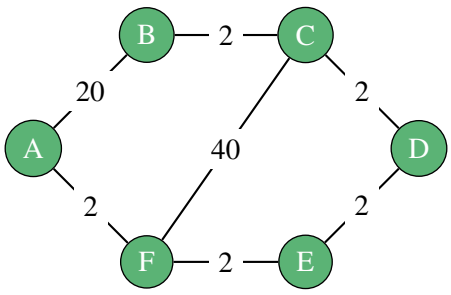
$\forall t$, calculer δ^{t+1} en fonction de δ^t ?

On a pour u, v, un plus court chemin de longueur au plus t.

Pour obtenir un plus court chemin de u à v en passant par au plus $t+1$ arêtes, il suffit de prendre $\min\{\delta^t(u\,v), \min\{\delta^t(u\,x) + w(x, v)\}\}$ x voisins de v.

On peut s'arrêter lorsque $t = n - 1$ (après on forme des cycles).

On note `Pred[u][v]` le prédécesseur de v dans le plus court chemin de u à v.



D

	A	B	C	D	E	F
A	0	20	-	-	-	2
B	20	0	2	-	-	-
C	-	2	0	2	-	40
D	-	-	2	0	2	-
E	-	-	-	2	0	2
F	2	-	40	-	2	0

Pred

	A	B	C	D	E	F
A	A	A	-	-	-	A
B	A	B	B	-	-	-
C	-	B	C	C	-	C
D	-	-	C	D	D	-
E	-	-	-	D	E	E
F	A	-	C	-	E	F

$t = 1$

	A	B	C	D	E	F
A	0	20	22	-	-	2
B	20	0	2	4	-	22
C	22	2	0	2	4	40
D	-	4	2	0	2	4
E	-	-	4	2	0	2
F	2	22	40	4	2	0

	A	B	C	D	E	F
A	A	A	B	-	-	A
B	A	B	B	C	-	A
C	B	B	C	C	D	C
D	-	C	C	D	D	E
E	-	-	D	D	E	E
F	A	A	C	E	E	F

$t = 2$

etc.

Pour trouver le plus court chemin de C à F il faut attendre $t = 3$, de AB, $t = 5 = n - 1$.

Implémentation

Dans l'implémentation de l'algorithme il va falloir faire des copies, sinon à l'instant t, `D` contient un mélange de δ^t et δ^{t+1}.

```
from utils import dcopy
def PCC1(G, w):
    inf = float("inf")
    D = {u:{u:inf for u in G.sommets} for u in G.sommets}
    Pred = {u:{u:None for u in G.sommets} for u in G.sommets}
    for u in G.sommets:
        for v in G.sommets:
            D[u][v] = w[u][v]
            Pred[u][v] = None if w[u][v] == inf else u
    D_next = dcopy(D)
    Pred_next = dcopy(Pred)
    for t in range(2, len(G.sommets)):
        for u in G.sommets:
            for v in G.sommets:
                for x in v.voisins:
                    if D_next[u][v] > D[u][x] + w[x][v]:
                        D_next[u][v] = D[u][x] + w[x][v]
                        Pred_next[u][v] = x
        D = dcopy(D_next)
        Pred = dcopy(Pred_next)
    return (D, Pred)
```

11.3 En doublant la longueur des chemins à chaque itération

Idée : Au temps t, calculer $\delta^{2^t}(u,v)\ \forall u,v$ en utilisant $\delta^{2^{t-1}}(u,v)\ \forall u,v$.
Initialement : $\delta^{2^t}(u,v) = w(u,v)\ \forall u,v$
$\forall t$, calculer $\delta^{2^{t+1}}$ en fonction de $\delta^{2^{t+1}}$?

$\forall u$, un plus court chemin de longueur au plus $2^{t+1} = 2 \times 2^t$ arêtes est composé de 2 plus courts chemins de longueur au plus 2^t .

$$u \xrightarrow{\leq 2^t} x \xrightarrow{\leq 2^t} v$$

Pour obtenir un plus court chemin de u à v en passant par au plus 2^{t+1} arêtes il suffit de prendre :

$$\min\{\delta^{2^t}(u,x), \min_{x \in S}\{\delta^{2^t}(u,x) + \delta^{2^t}(x,v)\}\}$$

On peut s'arrêter lorsque $\delta^{2^t} \geq n-1$.

Implémentation

On note `Pred[u][v]` le prédécesseur de v dans le plus court chemin de u à v.

```
1  from utils import dcopy
2  from math import log, ceil
3  def PCC1(G, w):
4      inf = float("inf")
5      D = {u:{u:inf for u in G.sommets} for u in G.sommets}
6      Pred = {u:{u:None for u in G.sommets} for u in G.sommets}
7      for u in G.sommets:
8          for v in G.sommets:
9              D[u][v] = w[u][v]
10             Pred[u][v] = None if w[u][v] == inf else u
11     D_next = dcopy(D)
12     Pred_next = dcopy(Pred)
13     for t in range(2, ceil(log(len(G.sommets) - 1, 2))):
14         for u in G.sommets:
15             for v in G.sommets:
16                 for x in G.sommets:
17                     if D_next[u][v] > D[u][x] + w[x][v]:
18                         D_next[u][v] = D[u][x] + w[x][v]
19                         Pred_next[u][v] = Pred[x][v]
20         D = dcopy(D_next)
21         Pred = dcopy(Pred_next)
22     return (D, Pred)
```

11.4 Algorithme de Floyd-Warshall

Idée : Supposons que les sommets soient numérotés de 1 à n.

À chaque itération t, on va calculer la longueur des plus courts chemins de u à v $\forall u, v \in S$ qui passent uniquement par les sommets $\{1, ..., t\}$

Appelons $\delta^R_{R \subseteq S}(u,x)$ la longueur du plus court chemin de u à v passant uniquement par des sommets appartenant à R.

De façon générale, si on a calculé $\forall u, v \;\; \delta^{1,\dots,k}(u,v)$

$$\delta^{1,\dots,k}(u,v) = \min\{\delta^{1,\dots,k}(u,v), \delta^{1,\dots,k}(u,k+1)\delta^{1,\dots,k}(k+1,v)\}$$

	A	B	C	D	E	F
A	0	20	-	-	-	2
B	20	0	2	-	-	-
C	-	2	0	2	-	40
D	40	-	2	0	2	-
E	-	-	-	2	0	2
F	2	-	-	-	2	0

	A	B	C	D	E	F
A	-	-	-	-	-	-
B	-	-	-	-	-	-
C	-	-	-	-	-	-
D	-	-	-	-	-	-
E	-	-	-	-	-	-
F	-	-	-	-	-	-

$t = 1$: $\delta(B,F) = 22$
$t = 2$: $\delta(A,C) = 22$
$\delta(C,F) = 26$

$t = 3$: $\delta(A,D) = \delta(A,C) + \delta(C,D) = 22+2 = 24$
$\delta(D,F) = \delta(D,C) + \delta(C,F) = 2+24 = 26$
$\delta(B,D) = \delta(B,C) + \delta(C,D) = 2+2 = 4$

Implémentation

```
from utils import dcopy
def Floyd_Warshall(G, w):
    inf = float("inf")
    D = {u:{u:inf for u in G.sommets} for u in G.sommets}
    Pred = {u:{u:None for u in G.sommets} for u in G.sommets}
    for u in G.sommets:
        for v in G.sommets:
            D[u][v] = w[u][v]
            Pred[u][v] = None if w[u][v] == inf else u
    D_next = dcopy(D)
    Pred_next = dcopy(Pred)
    for x in G.sommets:
        for u in G.sommets:
            for v in G.sommets:
                    if D[u][x] + D[x][v] < D_next[u][v]:
                        D_next[u][v] = D[u][x] + D[x][v]
                        Pred_next[u][v] = Pred[x][v]
        D = dcopy(D_next)
        Pred = dcopy(Pred_next)
    return (D, Pred)
```

12. Cycles négatifs

Définition 12.0.1 — Cycle négatif. Un cycle $\mathscr{C} = \{u_1, \dots, u_k$ est négatif si $\sum_{i=1}^{k-1} w(u_i, ui+1) + w(u_k, u_1 < 0$. Pour simplifier les équations, on écrit $W_{\mathscr{C}} = \sum_{(u,v)\in\mathscr{C}} w(u,v)$

Remarque : l'algorithme de Dijkstra ne marche pas pour les graphes avec des poids négatifs.

12.1 Algorithme pour détecter les cycles négatifs

Supposons qu'on vient d'exécuter Bellman-Ford et qu'on a `dist[s][v]` $\forall v$. On exécute alors un tour de boucle supplémentaire, si on découvre un meilleur chemin (ce qui n'est normalement pas possible, car on a trouvé tous les plus courts chemins), c'est qu'il y a un cycle négatif.

```
1 def CycleNegatif(G, w):
2     s = G.sommets[0] #un sommet de depart
3     Bellman_Ford(G, w, s)
4     for u in G.sommets:
5         for v in u.voisins:
6             if v.dist > u.dist + w[(u, v)]:
7                 return True
8     return False
```

Complexité : Bellman-Ford + $O(n+m)$

Index

ACM, 37
Algorithme de Kruskal, 37
Algorithme de Prim, 39
Arbre, 10
Arbre binaire, 21
Arbre binaire de recherche, 21
Arbre couvrant, 10, 11
Arbre couvrant minimal, 37
Arbre de parcours, 32

Bellman-Ford (algorithme), 45

Chemin, 10
Chemin simple, 10
Chemins, 31, 33
Compatibilité, 37
Complexité, 21
Complexité au pire cas, 21
Complexité en moyenne, 21
Composante connexe, 13
Connexe, 10
Coût amorti, 21
Cycle, 10
Cycle négatif, 51

Dijkstra, 41

FIND, 15
Floyd-Warshall (Algorithme), 49

Graphe, 9
Graphe des prédécesseurs, 32
Graphe pondéré, 11, 37
Graphe valué, 11, 37

Hauteur (Arbre), 22

Insertion (ABR), 23

Kruskal, 37

Liste d'adjacence, 10

Matrice d'adjacence, 10

Parcours en largeur, 28, 31
Parcours en profondeur, 33
Prim, 39
Problème, 11
Profondeur d'un nœud (Arbre), 22

Recherche dans un ABR, 22

Structure de données, 11

UNION, 15